

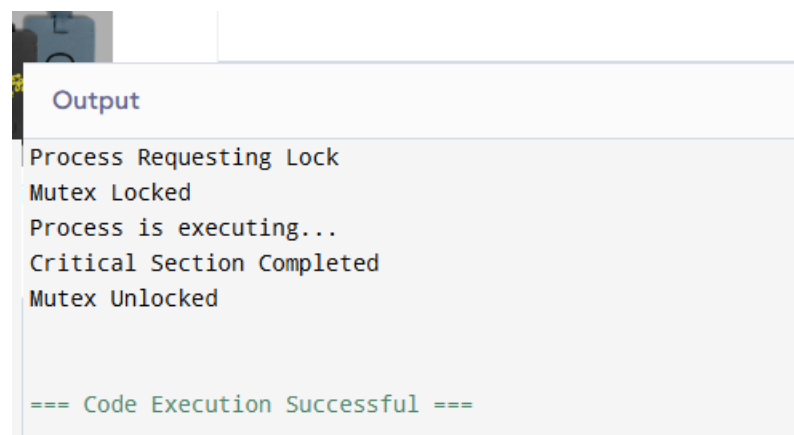
19. Design a C program to implement process synchronization using mutex locks.

Code:

A screenshot of a web-based C compiler interface. The editor shows a C program for process synchronization using a mutex lock. The code includes a header file, declares a lock variable, defines a critical section function, and implements a main function that requests the lock, enters the critical section, and then unlocks it. The interface includes a 'Run' button and a file explorer on the left.

```
1 #include <stdio.h>
2
3 int lock = 0;
4
5 void criticalSection() {
6     printf("Process is executing...\n");
7     printf("Critical Section Completed\n");
8 }
9
10 int main() {
11     printf("Process Requesting Lock\n");
12
13     if(lock == 0) {
14         lock = 1;
15         printf("Mutex Locked\n");
16
17         criticalSection();
18
19         lock = 0;
20         printf("Mutex Unlocked\n");
21     }
22     else {
23         printf("Resource Busy\n");
24     }
25
26     return 0;
27 }
```

Output:

A screenshot of the output window of the online compiler. It displays the execution results of the C program, showing the sequence of messages from the printf statements. The output confirms that the process successfully requested the lock, entered the critical section, and then unlocked it. The window is titled 'Output' and ends with a success message.

```
Process Requesting Lock
Mutex Locked
Process is executing...
Critical Section Completed
Mutex Unlocked

=== Code Execution Successful ===
```